

Aula 11 - Criptografia Assimétrica

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
October 11, 2025

A criptografia de chave pública introduz uma mudança fundamental:

- Baseada em **funções matemáticas**, e não apenas em substituição e permutação.
- É **assimétrica**: utiliza duas chaves separadas (pública e privada), ao contrário da simétrica que usa uma única chave.
- Essa assimetria impacta diretamente a **confidencialidade**, a **distribuição de chaves** e a **autenticação**.
- Grande parte da teoria dos criptossistemas de chave pública é baseada na teoria dos números.

Concepções equivocadas comuns:

- Não é intrinsecamente mais segura que a criptografia simétrica — a segurança depende do **tamanho da chave** e do **custo computacional** para quebrar a cifra.
- Não substitui a criptografia simétrica — mas apresenta toda uma gama de aplicações:
 - **Gerenciamento de chaves**
 - **Assinaturas digitais**

Chaves Assimétricas: Duas chaves relacionadas, uma **pública** e uma **privada**, usadas para operações complementares:

- Encriptação e deciptação
- Geração e verificação de assinatura

Certificado de Chave Pública: Documento emitido e assinado digitalmente pela chave privada de uma **Autoridade de Certificação (CA)**:

- Vincula o **nome de um assinante** a uma **chave pública**.
- Garante que o assinante identificado tem **controle exclusivo** da chave privada correspondente.

Algoritmo Criptográfico Assimétrico: Algoritmo criptográfico que usa duas chaves relacionadas:

- Uma chave **pública** e uma chave **privada**.
- É computacionalmente inviável derivar a chave privada a partir da pública.

Infraestrutura de Chave Pública (PKI): Conjunto de políticas, processos e plataformas para administração de certificados e pares de chaves:

- Emissão, manutenção e revogação de certificados.
- Envolve servidores, software e estações de trabalho.
- Suporte para autenticação, confidencialidade e integridade.

Como Surgiu a Criptografia Assimétrica

O conceito de criptografia de chave pública evoluiu de uma tentativa de atacar **dois dos problemas mais difíceis associados à encriptação simétrica**:

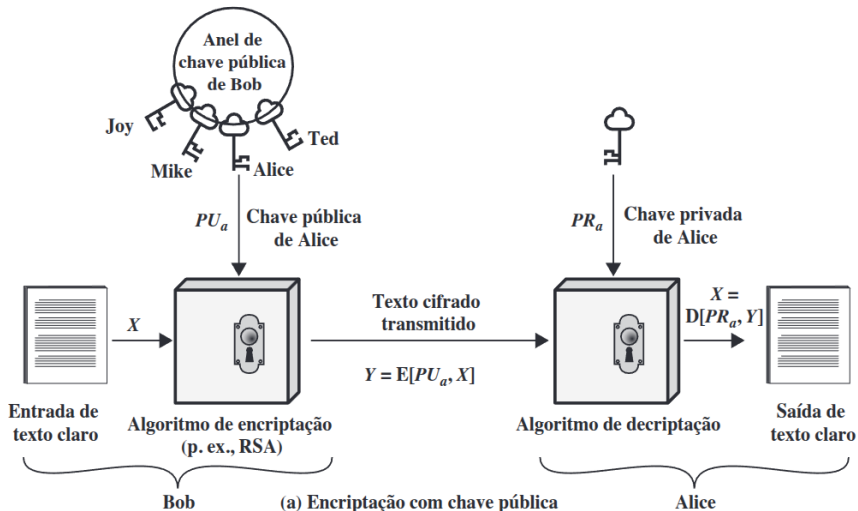
- **A distribuição de chaves** sob a encriptação simétrica:
 - Requer que dois comunicantes já compartilhem uma chave, que de alguma forma foi distribuída a eles; **OU**
 - Requer o uso de um centro de distribuição de chaves - Diffie [DIFF88]: “afinal, qual seria a vantagem de desenvolver criptossistemas impenetráveis, se seus usuários fossem forçados a compartilhar suas chaves com um CDC que poderia ser comprometido por roubo ou suborno?”
- **Assinaturas digitais** - Se o uso da criptografia tivesse que se tornar comum, não apenas nas situações militares, mas para fins comerciais e particulares, então as mensagens e documentos eletrônicos precisariam do equivalente das assinaturas usadas nos documentos em papel.

Os algoritmos assimétricos contam com uma chave para encriptação e uma chave diferente, porém relacionada, para a decifração. Eles têm a seguinte característica importante:

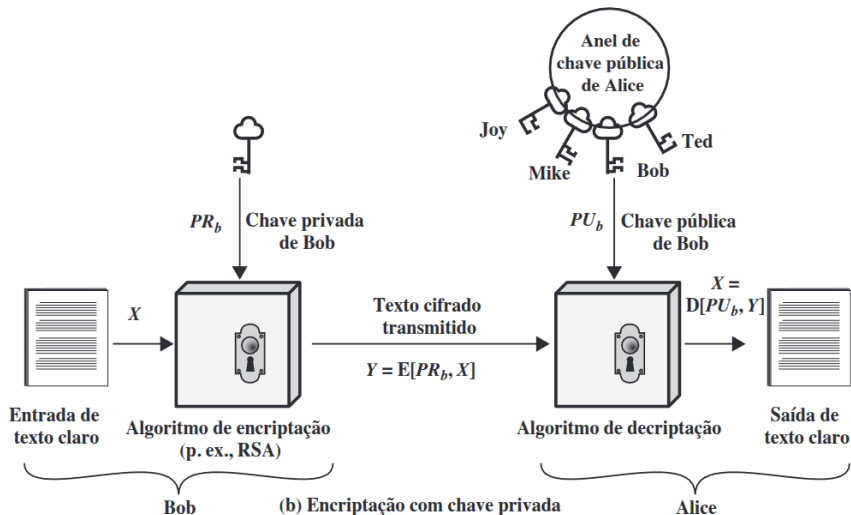
- É computacionalmente inviável determinar a chave de decifração dado apenas o conhecimento do algoritmo de criptografia e da chave de encriptação.
- Além disso, alguns algoritmos, como RSA, também exibem esta característica: Qualquer uma das duas chaves relacionadas pode ser usada para encriptação, com a outra para a decifração.

Encriptação com chave pública

Figura 9.1 Criptografia de chave pública.



Encriptação com chave privada



Etapas Essenciais da Criptografia de Chave Pública

1. Cada usuário gera um **par de chaves** (pública e privada) para encriptação e deciptação.
2. A **chave pública** é disponibilizada em um repositório ou arquivo acessível; a **chave privada** permanece secreta.
3. Para enviar uma mensagem confidencial a Alice, Bob usa a **chave pública de Alice** para encriptar a mensagem.
4. Ao receber, Alice utiliza sua **chave privada** para deciptar. Apenas ela consegue recuperar o texto original.

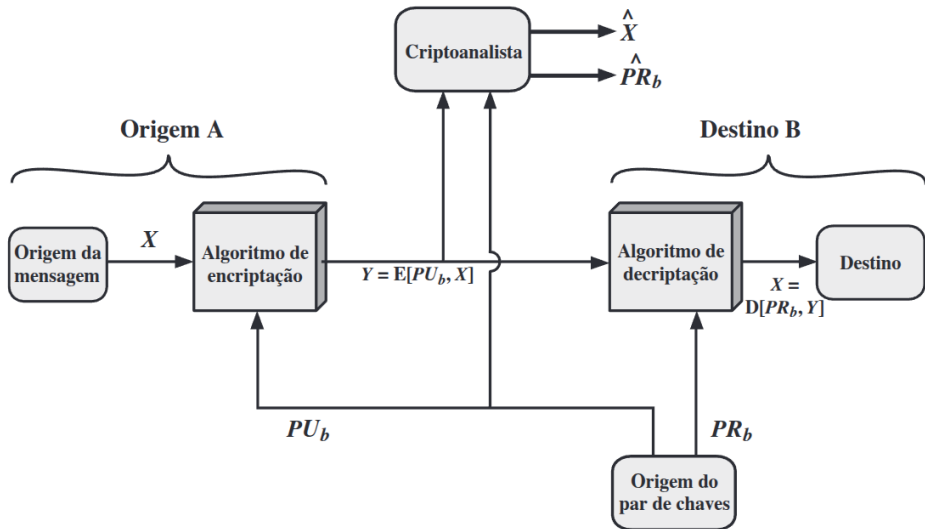
Esquema de Encriptação de Chave Pública

Um esquema de encriptação de chave pública possui cinco elementos principais:

1. **Texto claro:** mensagem ou dados legíveis fornecidos como entrada.
2. **Algoritmo de encriptação:** aplica transformações no texto claro.
3. **Chaves pública e privada:** par de chaves; se uma encripta, a outra decripta.
4. **Texto cifrado:** mensagem embaralhada gerada como saída, dependente da chave.
5. **Algoritmo de decriptação:** reverte o processo, recuperando o texto claro original.

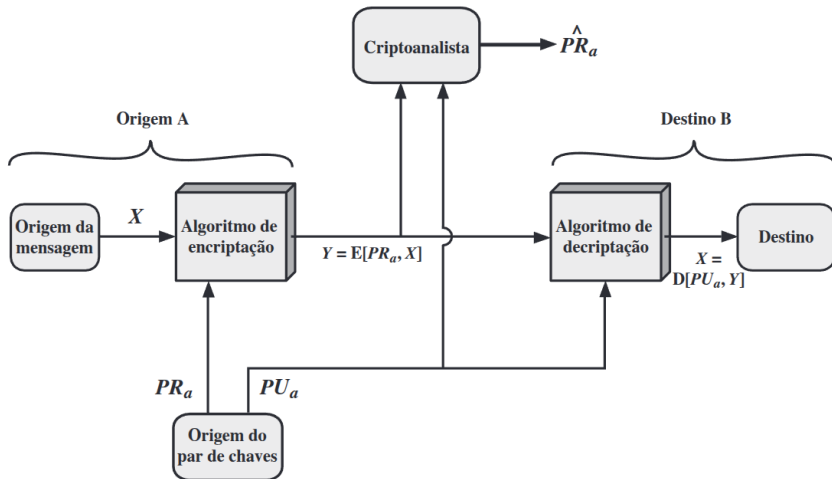
Criptossistema de chave pública - Objetivo Sigilo

Figura 9.2 Criptossistema de chave pública: sigilo.



Criptossistema de chave pública - Objetivo autenticação

Figura 9.3 Criptossistema de chave pública: autenticação.



Autenticação e Confidencialidade com Chave Pública

Importante:

- Apenas a encriptação com a chave privada do emissor garante **autenticação**, mas não a confidencialidade.
- Para prover **ambos**, aplica-se uma dupla encriptação:

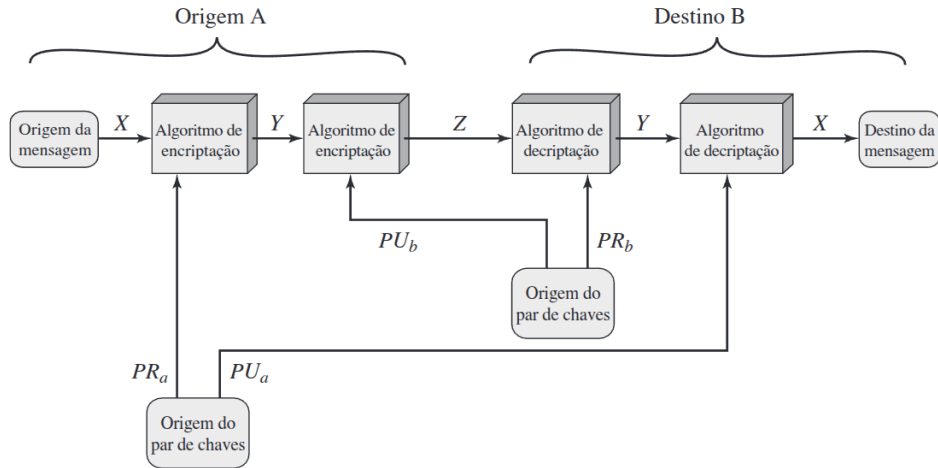
$$Z = E(PU_b, E(PR_a, X))$$

$$X = D(PU_a, D(PR_b, Z))$$

- O emissor primeiro encripta com sua **chave privada** (assinatura digital).
- Em seguida, encripta novamente com a **chave pública do receptor**.
- O receptor, ao decifrar, recupera tanto a **confidencialidade** quanto a **autenticidade**.
- Desvantagem: exige 4 operações de criptografia assimétrica por mensagem.

Criptossistema de chave pública - Objetivo autenticação e sigilo

Figura 9.4 Criptossistema de chave pública: autenticação e sigilo.



Aplicações de criptografia de chave pública

Quadro 9.3 Aplicações para criptossistemas de chave pública.

ALGORITMO	ENCRIPTAÇÃO/ DECRIPTAÇÃO	ASSINATURA DIGITAL	TROCA DE CHAVE
RSA	Sim	Sim	Sim
Curva elíptica	Sim	Sim	Sim
Diffie-Hellman	Não	Não	Sim
DSS	Não	Sim	Não

Obs: DSS é a sigla para Digital Signature Standard, um padrão de assinatura digital que, em conjunto com o DSA (Digital Signature Algorithm) e o SHA (Secure Hash Algorithm), permite a criação de assinaturas digitais e verificação de integridade de dados, utilizando criptografia de chave pública de forma a garantir a autenticidade e a não-repudição de uma transação

Requisitos para Criptografia de Chave Pública

1. Geração de par de chaves (PU_b, PR_b) deve ser **computacionalmente fácil**.
2. Encriptação: dado PU_b e M , calcular $C = E(PU_b, M)$ deve ser **fácil**.
3. Decriptação: dado PR_b e C , calcular $M = D(PR_b, C)$ deve ser **fácil**.
4. **Inviável** obter PR_b a partir de PU_b .
5. **Inviável** recuperar M conhecendo apenas PU_b e C .
6. (Opcional) Ordem das chaves pode ser invertida:
$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)].$$

Observação: Apenas poucos algoritmos atendem a todos os requisitos, como RSA, ECC, Diffie-Hellman e DSS.

Função de Mão Única com Alçapão

- Função de mão única: mapeia um domínio X em um intervalo Y de forma que:

$Y = f(X)$ é fácil de calcular, mas $X = f^{-1}(Y)$ é inviável de obter.

- "Com alçapão": existe uma informação secreta que permite inverter a função de forma eficiente.
- Essencial para criptografia de chave pública: permite encriptação segura sem compartilhar a chave privada.

Complexidade de Funções Criptográficas

- **Fácil:** problema resolvível em tempo polinomial $O(n^a)$ em função do tamanho da entrada n . Onde a é uma constante fixa.
- Algoritmos polinomiais pertencem à classe P.
- **Inviável:** problema cujo esforço cresce mais rápido que o tempo polinomial, por exemplo, $O(2^n)$.
- Observação: medir complexidade pelo pior caso ou caso médio não é suficiente para criptografia.
- Criptografia exige que a função seja inviável de reverter para **praticamente todas as entradas**.

Função de Mão Única com Alçapão

- Uma função que é fácil de calcular em uma direção e inviável na outra, a menos que informação adicional (alçapão) seja conhecida.
- Formalmente, uma família de funções reversíveis f_k :
 - $Y = f_k(X)$ é fácil, se k e X forem conhecidos.
 - $X = f_k^{-1}(Y)$ é fácil, se k e Y forem conhecidos.
 - $X = f_k^{-1}(Y)$ é inviável, se apenas Y for conhecido, sem k .
- A construção de esquemas de **chave pública** depende de **encontrar funções de mão única com alçapão adequadas**.

- O artigo de Diffie e Hellman (1976) desafiou a comunidade a criar algoritmos de chave pública. [Referência](#)
- Muitos algoritmos iniciais se mostraram falhos.
- Em 1977, Ron Rivest, Adi Shamir e Len Adleman (MIT) desenvolveram o esquema RSA. [Referência](#)
- Publicado em 1978, o RSA se tornou a técnica mais aceita e amplamente implementada para criptografia de chave pública.

Funcionamento do RSA

- RSA é uma cifra de bloco: o texto claro e o texto cifrado são inteiros entre 0 e $n - 1$.
- Tamanho típico: $n \approx 1024$ bits (cerca de 309 dígitos decimais).
- Cada bloco de texto claro M deve satisfazer $0 \leq M < n$.
- Encriptação e deciptação:

$$C = M^e \bmod n,$$

$$M = C^d \bmod n = (M^e)^d \bmod n$$

- Chave pública: $PU = \{e, n\}$
Chave privada: $PR = \{d, n\}$
- Requisitos:
 1. $M^{ed} \bmod n = M$ para todo $M < n$
 2. Calcular $M^e \bmod n$ e $C^d \bmod n$ é eficiente
 3. Conhecendo apenas e e n , é inviável determinar d

Requisito de Inversão no RSA

- Para garantir que $M^{ed} \bmod n = M$, é necessário que:
 - os expoentes e e d sejam **inversos multiplicativos módulo** $\varphi(n)$,
 - onde $\varphi(n)$ é a **função totiente de Euler**.
- Para $n = pq$, com p e q primos:

$$\varphi(n) = (p - 1)(q - 1)$$

- Relação fundamental:

$$ed \equiv 1 \pmod{\varphi(n)} \Rightarrow d \equiv e^{-1} \pmod{\varphi(n)}$$

- O inverso multiplicativo **só existe** se e for **coprimo com** $\varphi(n)$:

$$\text{mdc}(e, \varphi(n)) = 1$$

- O valor de d pode ser obtido com o **algoritmo de Euclides estendido**, que encontra o inverso modular de e em relação a $\varphi(n)$.

Inverso Multiplicativo no RSA

No RSA, a chave privada d é escolhida como o inverso multiplicativo de e módulo $\varphi(n)$:

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

- Isso significa que d satisfaz:

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

- Exemplo: $e = 7$, $\varphi(n) = 160$

$$23 \cdot 7 = 161 \equiv 1 \pmod{160}$$

- Portanto, $d = 23$ é o inverso multiplicativo de $e = 7$ módulo 160.

Entendendo como funciona: O que queremos garantir

No RSA:

- Mensagem M
- Chave pública (n, e)
- Chave privada (n, d)

Criptografia:

$$C \equiv M^e \pmod{n}$$

Descriptografia:

$$M' \equiv C^d \pmod{n} = (M^e)^d \pmod{n}$$

Objetivo:

$$M' \equiv M \pmod{n}$$

Entendendo como funciona: Por que e e d devem ser inversos módulo $\varphi(n)$

- $n = pq$, $\varphi(n) = (p - 1)(q - 1)$
- Teorema de Euler: $M^{\varphi(n)} \equiv 1 \pmod{n}$ se $\text{mdc}(M, n) = 1$
- Escolhemos d tal que:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

- Ou seja, existe k tal que $ed = 1 + k\varphi(n)$

Entendendo como funciona: Aplicando na descryptografia

$$M' = C^d = (M^e)^d = M^{ed} = M^{1+k\varphi(n)} = M \cdot (M^{\varphi(n)})^k$$

Pelo Teorema de Euler:

$$(M^{\varphi(n)})^k \equiv 1^k \equiv 1 \pmod{n}$$

Portanto:

$$M' \equiv M \pmod{n}$$

Conclusão: funciona porque $e \cdot d \equiv 1 \pmod{\varphi(n)}$

Segurança do RSA

- **1. Estrutura das chaves:**

- Chave pública: $PU = \{e, n\}$
- Chave privada: $PR = \{d, n\}$
- Onde $n = p \cdot q$, e p e q são primos grandes e secretos

- **2. Se p e q forem descobertos:**

- É possível calcular o totiente de Euler:

$$\varphi(n) = (p - 1)(q - 1)$$

- E então obter o inverso modular de e :

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

- Logo, o atacante consegue reconstruir a chave privada d

- **3. Conclusão:**

- A segurança do RSA depende da dificuldade de **fatorar** n .
- Se alguém conseguir obter p e q , o sistema é completamente quebrado.
- Portanto, manter p e q secretos é **fundamental**.

Escolha do valor e no RSA

- **Geralmente, escolhemos** $e = 65537 = 2^{16} + 1$
- É considerado o “**ponto ideal**” entre segurança e desempenho.

Propriedades:

- **Ímpar**: requisito para ser coprimo com $\varphi(n)$ na maioria dos casos.
- **Pequeno o bastante para ser rápido**:
 - Tem apenas 17 bits.
 - Exige pouquíssimas multiplicações na exponenciação modular.
- **Grande o suficiente para ser seguro**:
 - Evita ataques que exploram expoentes muito pequenos (como $e = 3$ ou $e = 17$).
- **Forma binária simples**:

$$e = 10000000000000001_2$$

o que torna a operação de exponenciação modular bem eficiente.

Esquema RSA

Ingredientes:

- Dois números primos secretos: p e q .
- Calcula-se $n = pq$ (público).
- Escolhe-se e tal que $\text{mdc}(\varphi(n), e) = 1$ e $1 < e < \varphi(n)$ (público).
- Calcula-se $d \equiv e^{-1} \pmod{\varphi(n)}$ (privado).

Chaves:

- Chave privada: $\{d, n\}$
- Chave pública: $\{e, n\}$

Processo de encriptação e deciptação:

- Usuário B quer enviar mensagem M para A:

$$C = M^e \pmod{n} \quad (\text{encriptação})$$

- Usuário A recebe C e decipta:

$$M = C^d \pmod{n}$$

Tabela resumo dos parâmetros do RSA

Figura 9.5 O algoritmo RSA.

Geração de chave por Alice	
Selecione p, q	p e q são primos, $p \neq q$
Calcule $n = p \times q$	
Calcule $\phi(n) = (p - 1)(q - 1)$	
Selecione o inteiro e	$\text{mdc}(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcule d	$d \equiv e^{-1} \pmod{\phi(n)}$
Chave pública	$PU = \{e, n\}$
Chave privada	$PR = \{d, n\}$
Encriptação por Bob com chave pública de Alice	
Texto claro:	$M < n$
Texto cifrado:	$C = M^e \pmod{n}$
Deciptação por Alice com a chave privada de Alice	
Texto cifrado:	C
Texto claro:	$M = C^d \pmod{n}$

Exemplo Numérico do RSA

Geração de chaves:

- $p = 17, q = 11$
- $n = pq = 187$
- $\varphi(n) = (p - 1)(q - 1) = 160$
- Escolhe $e = 7, \text{mdc}(e, 160) = 1$
- Calcula $d \equiv e^{-1} \pmod{160} \Rightarrow d = 23$

Explicação

- Exemplo: $e = 7, \varphi(n) = 160$

$$23 \cdot 7 = 161 \equiv 1 \pmod{160}$$

- Portanto, $d = 23$ é o inverso multiplicativo de $e = 7$ módulo 160.

Chaves:

- Pública: $PU = \{7, 187\}$
- Privada: $PR = \{23, 187\}$

Exemplo Numérico do RSA

Encriptação de $M = 88$:

$$C = M^e \mod n = 88^7 \mod 187$$

$$88^7 \mod 187 = (88 \times 88^2 \times 88^4) \mod 187 = (88 \times 77 \times 132) \mod 187 = 11$$

Deciptação:

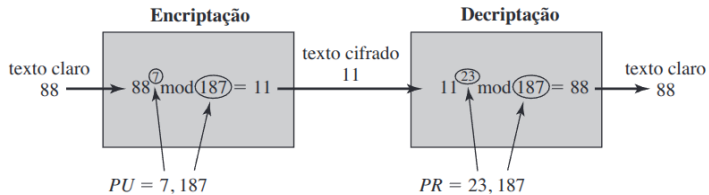
$$M = C^d \mod n = 11^{23} \mod 187$$

$$11^{23} \mod 187 = 88$$

Resultado: A mensagem original $M = 88$ é recuperada com sucesso.

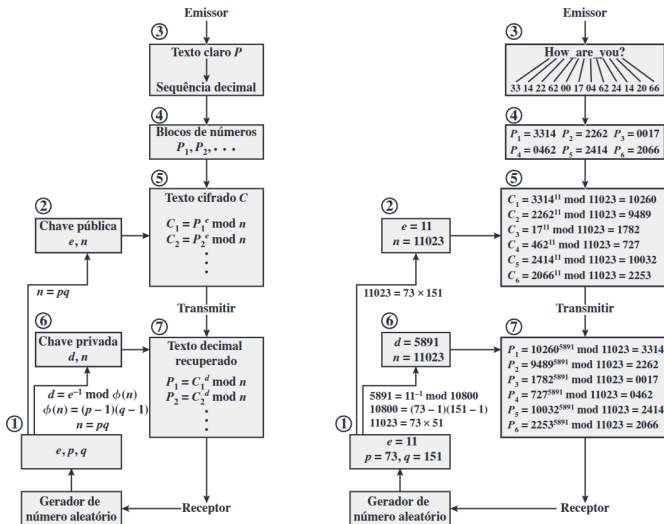
Exemplo Simples numérico - RSA

Figura 9.6 Exemplo de algoritmo RSA.



Processamento de múltiplos blocos usando RSA

Figura 9.7 Processamento de múltiplos blocos com RSA.



Usando e verificando os parâmetros RSA na prática

- **Gerar a chave ssh:**

```
ssh-keygen -t rsa -f ./teste
```

- **Verificar fingerprint da chave:**

```
ssh-keygen -lf ./teste
```

- **Tentear abrir** (não vai funcionar):

```
openssl rsa -in ./teste -text -noout
```

- **Mudar o formato:**

```
ssh-keygen -p -m PEM -f ./teste
```

- **Verificar fingerprint anterior:**

```
ssh-keygen -lf ./teste
```

- **Tentear abrir** (vai funcionar):

```
openssl rsa -in ./teste -text -noout
```

- **Mudar formato da chave pública para modo PEM:**

```
ssh-keygen -e -m PEM -f ./teste.pub > ./teste_pub.pem
```

- **Abrir a chave publica:**

```
openssl rsa -pubin -in teste_pub.pem -text -noout
```

O que é cada parâmetro de uma chave RSA

Nome	Símbolo	Função
modulus	$n = p \cdot q$	Base da aritmética modular — o número grande
publicExponent	e	Expoente público, normalmente 65537.
privateExponent	d	Expoente privado usado para assinar/decifrar.
prime1	p	Primeiro primo secreto.
prime2	q	Segundo primo secreto.
exponent1 / exponent2	$d \bmod (p - 1)$ e $d \bmod (q - 1)$	Para otimização - Teorema Chinês do Resto.
coefficient	$q^{-1} \bmod p$	Outro parâmetro auxiliar do Teo. Chin. Resto.

O Problema da Fatoração no RSA

A segurança do RSA depende da dificuldade de certos problemas matemáticos. Existem três possíveis formas de ataque:

1. **Fatorar n :** encontrar os primos p e q tais que $n = p \times q$. Isso permite calcular $\varphi(n) = (p - 1)(q - 1)$ e, em seguida, $d \equiv e^{-1} \pmod{\varphi(n)}$.
2. **Determinar $\varphi(n)$ diretamente:** sem fatorar n , mas que leva igualmente à descoberta de d .
3. **Descobrir d diretamente:** a partir de e e n , sem precisar de $\varphi(n)$. Até hoje, parece tão difícil quanto fatorar n .

Conclusão:

- A criptoanálise do RSA está fortemente ligada ao problema da **fatoração de inteiros**.
- O desempenho dos melhores algoritmos de fatoração é usado como **benchmark de segurança**.

Dificuldade de Fatoração em Diferentes Tamanhos de Chave RSA

- A dificuldade de fatorar cresce rapidamente com o tamanho do módulo:
 - Fatorar um módulo RSA de **1024 bits** é cerca de **mil vezes mais difícil** do que um de 768 bits.
 - Um módulo de **768 bits** é milhares de vezes mais difícil que um de 512 bits.
- A primeira fatoração bem-sucedida de um módulo de **512 bits** ocorreu há cerca de uma década.
- É plausível que um módulo de **1024 bits** possa ser fatorado dentro da próxima década por grupos acadêmicos.

Recomendação prática:

- Evitar o uso de chaves RSA de 1024 bits nos próximos **3 a 4 anos**.
- Preferir módulos de pelo menos **2048 bits** para segurança a médio prazo.

Troca de Chaves Diffie-Hellman

- Primeiro algoritmo de chave pública, proposto por Diffie e Hellman em 1976.
- Objetivo: permitir que dois usuários troquem **variáveis** para criar uma chave secreta com segurança.
- Essa chave secreta pode ser usada posteriormente para criptografar mensagens.
- O algoritmo em si não criptografa mensagens, apenas realiza a **troca de valores secretos**.
- Baseado em propriedades da aritmética modular e do problema do logaritmo discreto.

Logaritmo Discreto e Diffie-Hellman

- A segurança do algoritmo Diffie-Hellman depende da dificuldade de calcular **logaritmos discretos**.
- Definição: dado um número primo p e uma raiz primitiva a , temos:

$$a^1 \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

que geram todos os inteiros de 1 até $p - 1$.

- Para qualquer inteiro b , existe um único expoente i tal que:

$$b \equiv a^i \pmod{p}, \quad 0 \leq i \leq p - 1$$

- Esse i é o **logaritmo discreto** de b na base a , módulo p :

$$i = \log_a(b) \pmod{p}$$

- Problema: calcular i é computacionalmente inviável para valores grandes de p .

Algoritmo de Troca de Chaves Diffie-Hellman

- Dois parâmetros públicos:

$$q \text{ (número primo)}, \quad \alpha \text{ (raiz primitiva de } q)$$

- Cada usuário escolhe um valor secreto:

$$A : X_A < q, \quad B : X_B < q$$

- Cada um calcula sua chave pública:

$$Y_A = \alpha^{X_A} \bmod q, \quad Y_B = \alpha^{X_B} \bmod q$$

- As chaves públicas Y_A e Y_B são trocadas.
- Cálculo da chave secreta compartilhada:

$$A : K = (Y_B)^{X_A} \bmod q$$

$$B : K = (Y_A)^{X_B} \bmod q$$

- Como $(Y_B)^{X_A} \equiv (Y_A)^{X_B} \pmod{q}$, ambos obtêm a mesma chave secreta K .

Calculo da chave K - Diffie-Hellman

A chave K é calculada por ambas as partes da seguinte forma:

$$\begin{aligned} K &= (Y_B)^{X_A} \bmod q \\ &= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\ &= (\alpha^{X_B})^{X_A} \bmod q \\ &= \alpha^{X_B X_A} \bmod q \\ &= (\alpha^{X_A})^{X_B} \bmod q \\ &= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\ &= (Y_A)^{X_B} \bmod q \end{aligned}$$

pelas regras da aritmética modular

Troca de Chaves Diffie-Hellman

O resultado é que os dois lados trocaram um valor secreto. Esse valor secreto é normalmente usado como chave secreta simétrica compartilhada.

Agora, considere um intruso que pode observar a troca de chaves e queira determinar a chave secreta K . Como X_A e X_B são privados, o intruso só tem os seguintes elementos para trabalhar: q , α , Y_A e Y_B .

Assim, o intruso é forçado a calcular um **logaritmo discreto** para estabelecer a chave. Por exemplo, para definir a chave privada do usuário B, o intruso precisaria calcular:

$$X_B = \log_{\alpha}(Y_B) \pmod{q}$$

Exemplo: Troca de Chaves Diffie-Hellman

A segurança da troca de chaves Diffie-Hellman está no fato de que, embora seja fácil calcular exponenciais módulo um primo, é muito difícil calcular logaritmos discretos. Para números grandes, esta tarefa é inviável.

Exemplo: $q = 353$, $\alpha = 3$ $X_A = 97$, $X_B = 233$

Cálculo das chaves públicas:

$$Y_A = 3^{97} \bmod 353 = 40 \quad Y_B = 3^{233} \bmod 353 = 248$$

Cálculo da chave secreta comum:

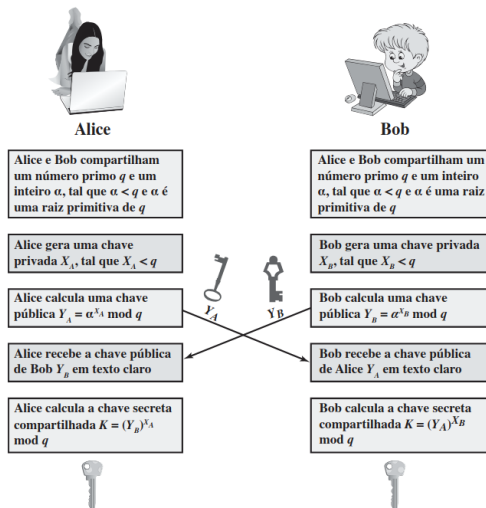
$$K_A = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$$

$$K_B = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$$

Para um intruso: Disponível apenas $q = 353$, $\alpha = 3$, $Y_A = 40$, $Y_B = 248$. Para descobrir K , teria de resolver um *logaritmo discreto*, o que é inviável para primos grandes.

Diffie-Hellman na prática

Figura 10.1 O algoritmo de troca de chaves Diffie-Hellman.



Troca simples (dois usuários):

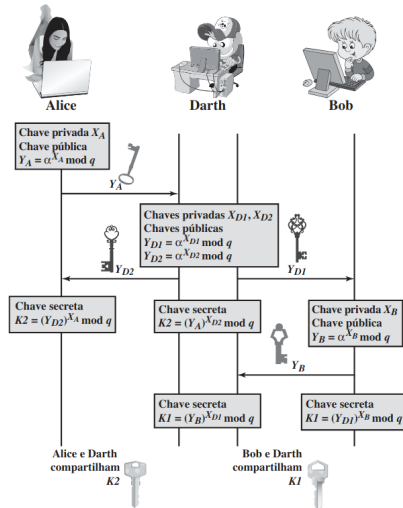
- A escolhe X_A , calcula $Y_A = \alpha^{X_A} \bmod q$ e envia a B .
- B escolhe X_B , calcula $Y_B = \alpha^{X_B} \bmod q$ e envia a A .
- Ambos calculam a chave secreta K :

$$K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q$$

- Os valores públicos q e α devem ser conhecidos previamente ou enviados na primeira mensagem.

Man-in-the-Middle - Diffie Hellman

Figura 10.2 Ataque *man-in-the-middle*.



Ataque Man-in-the-Middle em Diffie–Hellman

Consequência:

- Alice e Bob acreditam ter uma chave secreta compartilhada.
- Na prática, Alice compartilha K_2 com Darth e Bob compartilha K_1 com Darth.
- Darth pode **espiar** ou **modificar** mensagens.

Motivo da vulnerabilidade: O protocolo não autentica os participantes.

Solução: uso de **assinaturas digitais** e **certificados**.